

North East University Bangladesh

**Urban Mobility Demand Forecasting:
Time Series Prediction Using Deep Learning
on the Seoul Bike Sharing Dataset**

Submitted to:

Razorshi Prozzwal Talukder
Lecturer
Department of CSE
North East University Bangladesh

Submitted by:

Rohit Kumar Sen
ID: 0562220005101030
8th Semester
Department of CSE
North East University Bangladesh

Urban Mobility Demand Forecasting Using Deep Learning on Seoul Bike-Sharing Hourly Data

Rohit Kumar Sen

Department of Computer Science and Engineering
North East University Bangladesh

Abstract --

Accurate forecasting of hourly bike-sharing demand supports fleet rebalancing and operational planning in urban mobility systems. This study develops and compares deep learning models on the Seoul Bike Sharing dataset (2017-2018), treating demand prediction as a regression problem over 8,297 cleaned hourly records. A chronological 70/15/15 train-validation-test split and 24-hour input windows ensure evaluation on unseen future time. We implement Vanilla LSTM, Bidirectional LSTM, CNN-LSTM, regime-specific specialists, temporal attention, a 24-step encoder-decoder, and Keras Tuner optimization under a unified protocol. The tuned CNN-LSTM with raw weather features achieves the best one-step performance (RMSE 254.3 bikes, MAE 178.9, R^2 0.781). Raw rainfall, snowfall, and visibility outperform a composite severity index. Multi-step forecasting attains R^2 0.56 across 24 horizons. Results show that hybrid convolutional-recurrent models are well suited to this demand forecasting task.

Keywords -- bike sharing, demand forecasting, LSTM, CNN-LSTM, time series, deep learning, Seoul dataset.

1. Introduction

Shared bicycle systems reduce congestion and support short urban trips. For operators, the central operational question is how many bicycles will be needed in the next hour—and, for planning, how demand may evolve over the coming day. Forecasting errors lead to stockouts at stations or inefficient redistribution of idle bikes.

Traditional statistical methods (ARIMA, exponential smoothing) model temporal structure explicitly but struggle with nonlinear interactions between weather, calendar effects, and past demand. Deep learning sequence models can learn such patterns from data when sufficient history and features are available. This project applies modern recurrent and hybrid architectures to hourly Seoul bike-sharing records.

We formulate the problem as **regression**: the target variable is the continuous count of rented bicycles per hour. Performance is measured with RMSE and MAE (in bikes) and coefficient of determination (R^2). We do not use classification accuracy, as the output is not a discrete label.

1.1 Objectives

- Design a reproducible preprocessing and sequence-generation pipeline for hourly time series.
- Compare LSTM, BiLSTM, and CNN-LSTM baselines under identical training settings.
- Evaluate raw weather features versus an engineered weather severity index.

- Assess peak-hour and off-peak specialist models for heterogeneous demand regimes.
- Extend to attention-based and 24-hour encoder-decoder forecasting.
- Optimize the best architecture with automated hyperparameter search.

2. Dataset

The **Seoul Bike Sharing Demand** dataset is a publicly available collection of hourly observations spanning 2017-2018. Each record includes rental count, time of day, meteorological variables, and categorical attributes (season, holiday, system functioning status). The source file contains 8,760 hourly rows, yielding 8,297 samples after cleaning and feature engineering.

2.1 Variables and Features

The **target variable** is *Rented Bike Count*. **Meteorological predictors** include temperature, humidity, wind speed, visibility, solar radiation, rainfall, and snowfall. **Temporal features** include hour of day, day of week, month, and season. **Indicator variables** encode holidays and system functioning status. After feature engineering (lags, rolling statistics, and cyclic encodings), the modeling table expands to include lag features at 1-hour, 24-hour, and 168-hour (weekly) intervals, plus rolling means over 3-hour and 24-hour windows.

2.2 Temporal split strategy

Time series forecasting demands **chronological splitting** to prevent temporal leakage. We adopt a strict 70% train ($n=5,808$), 15% validation ($n=1,220$), and 15% test ($n=1,222$) split without shuffling. This ensures that all training occurs on historical data, validation tunes early stopping, and test evaluation occurs on truly unseen future time. With a 24-hour lookback window, one-step forecasting yields 1,222 sequence examples in the test set.

3. Preprocessing and Feature Engineering

Preprocessing transforms raw CSV data into normalized sequence tensors suitable for deep learning models. All transformation steps are applied in strict temporal order to prevent information leakage. Scalers and normalization statistics are computed exclusively from training data, ensuring that validation and test metrics reflect realistic generalization.

3.1 Cleaning and data quality

Records corresponding to non-functioning days are removed ($n<500$), as they represent system maintenance rather than operational demand. Extreme rental counts are capped at the 99th percentile to reduce the influence of rare spikes and one-off events. Rainfall and snowfall receive a $\log(1 + x)$ transform to handle right-skewed distributions. Dew point temperature is dropped due to high correlation ($r>0.95$) with humidity, reducing multicollinearity.

3.2 Feature engineering and representation

Calendar features: Day of week, month, and binary weekend indicators capture weekly and seasonal patterns. **Cyclic encoding:** Hour of day and month are encoded as sine-cosine pairs so that 23:00 and 00:00 are close in feature space, preserving circular structure. **Regime indicators:** A binary

peak-hour flag marks commute periods (07:00-09:00 and 17:00-20:00). **Lag and rolling features:** Lags at 1 h, 24 h, and 168 h (weekly) inject recent demand history; rolling means over 3-hour and 24-hour windows smooth noise. **Weather severity index:** A composite score combines independent meteorological signals for ablation experiments.

3.3 Normalization and windowing

Min-Max scaling maps all features and the target to [0, 1] using only training-set percentiles, preventing leakage. Each training/validation/test example is a tensor of shape (24, F)—24 consecutive hours with F features each. The label for one-step forecasting is the bike count at the next hour; for multi-step, labels comprise the next 24 hours. This windowing strategy maintains temporal contiguity and respects the chronological split.

4. Methodology

4.1 Experimental protocol

All experiments use **TensorFlow 2.21.0** with random seed 42 for reproducibility. Models train for up to 50 epochs with batch size 256, the **Adam optimizer** (learning rate 10^{-3} for baseline models, tuned for the best model), and **mean squared error (MSE) loss**. **Early stopping** monitors validation loss with patience 8, halting training when improvement plateaus. **ReduceLROnPlateau** reduces the learning rate by half when validation loss fails to improve for 3 epochs. Best model checkpoints are saved. All reported test metrics are computed after inverse scaling to original bike counts.

4.2 Feature ablation experiments

To isolate the contribution of weather representation, we train each architecture under two configurations: (i) **raw_weather**: Models use rainfall, snowfall, and visibility as separate channels, allowing the network to learn independent transformations. (ii) **severity_index**: These three features are replaced with a single engineered severity score. All other features, the temporal split, and training protocol remain constant, enabling direct comparison of weather representations.

4.3 Deep learning architectures

This section describes each model family in conceptual and implementation terms. Unless noted, all one-step models consume a 24-hour input window (shape (24, F)) and output a single prediction.

4.3.1 Vanilla LSTM

The **Long Short-Term Memory (LSTM)** network is a recurrent neural network designed to model sequential dependencies. An LSTM cell uses gating mechanisms—input, forget, and output gates—to decide what information to retain, update, or discard across time steps. This gating alleviates the vanishing gradient problem plaguing standard RNNs, making LSTMs effective for time series where both recent and moderately distant past values influence the future.

Implementation in this project. We implement a single LSTM layer with 64 hidden units, dropout rate 0.2, followed by a Dense layer with one output neuron for bike count. The network reads the full 24-hour window sequentially and produces the next-hour forecast.

Role in the experiment. Serves as the simplest deep learning baseline, establishing a lower bound against which more complex architectures are compared.

4.3.2 Bidirectional LSTM (BiLSTM)

A **Bidirectional LSTM** runs two LSTM layers on the same input sequence: one processes time forward, the other backward. Their outputs are concatenated so that each position depends on both earlier and later observations within the window. BiLSTMs often improve performance on tasks where full context is available, as they avoid the causality constraint of unidirectional models.

Implementation in this project. We wrap an LSTM(64) in a Bidirectional layer with dropout 0.2, then apply Dense(1). For a 24-hour history, the model can use information from hour 23 when interpreting hour 1, incorporating full-window context.

Role in the experiment. Tests whether access to both past and future context within the fixed 24-hour window improves demand forecasts. Note: true real-time forecasting would prefer unidirectional models, but this experiment validates the utility of full lookahead.

4.3.3 Convolutional LSTM (CNN-LSTM)

The **CNN-LSTM** hybrid combines one-dimensional convolution with recurrent modeling. A **Conv1D** layer scans the time axis with small learned filters (kernel size 3), detecting local patterns such as sudden weather changes or rapid demand shifts. **Causal padding** ensures each time position depends only on current and past inputs. Max pooling reduces temporal resolution and computational cost. The LSTM then aggregates the convolved feature maps into a fixed-size state vector for final prediction.

Implementation in this project. Architecture: Conv1D(64 filters, kernel 3, causal) → ReLU activation → MaxPool1D(2) → Dropout(0.2) → LSTM(64) → Dense(1). This design captures both local temporal patterns via convolution and long-range dependencies via recurrence.

Role in the experiment. Our primary baseline and the strongest model family in the comparison. Hybrid architectures combining convolution and recurrence have proven effective across many time series tasks, motivating this choice.

4.3.4 Peak and off-peak specialist models

Bike demand exhibits **strong heterogeneity across time of day**: morning (7-9 AM) and evening (5-8 PM) peaks reflect commuting and are highly predictable, while off-peak hours show smoother, weather-driven variation. A single global model must compromise between these regimes. **Specialist models** train separate CNN-LSTM networks on subsets of data where the target (label) hour falls within peak or off-peak windows.

Implementation in this project. We reuse the CNN-LSTM architecture but filter training, validation, and test sequences by the hour of the label, not the input window start. Peak specialist: sequences where the label hour is 7, 8, 9, 17, 18, 19, or 20 (n=357 test). Off-peak specialist: all other label hours (n=865 test).

Role in the experiment. Evaluates whether regime-specific models reduce error on their respective test subsets. Lower RMSE for specialists on their regimes would suggest that task-specific architectures outperform one-size-fits-all modeling.

4.3.5 CNN-LSTM with temporal attention

Attention mechanisms allow neural networks to focus adaptively on the most informative time steps. After the LSTM produces a hidden vector at each (downsampled) time position, a small feed-forward network computes attention scores. **Softmax** converts scores to weights summing to 1. The weighted sum of LSTM outputs forms the context vector feeding into the final Dense(1) prediction layer. Attention aids interpretability by revealing which past hours influenced the forecast.

Implementation in this project. Conv1D and pooling match the CNN-LSTM stack. LSTM uses `return_sequences=True` to output all time steps. Additive attention: a Dense(1) layer produces scores, Softmax produces weights, and context is a weighted sum. Context vector $\rightarrow$ Dense(1) $\rightarrow$ output.

Role in the experiment. Tests whether explicit temporal weighting improves accuracy over the simpler pooling-plus-LSTM design. Attention's interpretability may reveal whether the model focuses on recent vs. distant history or certain hours of the day.

4.3.6 Encoder-decoder LSTM (24-hour forecasting)

An **encoder-decoder** architecture maps a variable-length input sequence to a variable-length output. The **encoder** LSTM reads the 24-hour input and compresses it into a fixed-size state vector (hidden and cell states). The **decoder** LSTM is initialized with the encoder's final state and generates one output time step at a time. This architecture is well-suited for multi-step forecasting. Here, we forecast all 24 hours ahead in a single forward pass—harder than one-step prediction because errors can compound and long-range structure must be inferred.

Implementation in this project. Encoder: LSTM(64, `return_state=True`) consumes the 24-hour input and returns final hidden and cell states. RepeatVector(24) tiles the encoder output 24 times. Decoder: LSTM(64, `return_sequences=True`) initialized from encoder states, generates 24 time steps. TimeDistributed(Dense(1)) predicts one bike count per future hour.

Role in the experiment. Provides day-ahead demand trajectories for operational planning and scheduling. Multi-step RMSE is higher than one-step but enables 24-hour lookahead for fleet management.

4.3.7 Hyperparameter tuning (Keras Tuner)

Hyperparameter tuning automates the search for architectural and optimization settings that minimize validation loss. Manual tuning is time-consuming when many combinations exist (filter counts, kernel sizes, LSTM width, dropout, learning rate). **Random search** samples configurations uniformly from a defined space and trains each for a fixed budget. This often finds strong models more efficiently than exhaustive grid search in high-dimensional spaces.

Implementation in this project. Keras Tuner RandomSearch with `max_trials=10`, optimizing `val_loss`. Search space: `conv_filters` in {32, 64, 96}, `kernel_size` in {2, 3, 5}, `lstm_units` in {32, 64, 96}, `dropout` in [0.1, 0.5], `learning_rate` in {1e-3, 5e-4, 1e-4}. The best trial (64 filters, kernel 3, 96 LSTM units, dropout 0.2, $lr=5 \times 10^{-4}$) was fine-tuned for 50 epochs.

Role in the experiment. Produces `cnn_lstm_tuned_raw_weather`, our recommended one-step model. Automated tuning balances exploration and exploitation, avoiding manual bias toward familiar architectures.

4.4 Evaluation metrics

RMSE (Root Mean Squared Error) reports typical prediction error in bikes per hour, penalizing large mistakes more heavily. **MAE** (Mean Absolute Error) is the average absolute deviation in bikes, less sensitive to outliers. **R^2** (coefficient of determination) measures the proportion of target variance explained by the model. **sMAPE** (symmetric mean absolute percentage error) provides a scale-free metric more stable than MAPE when counts are low. All metrics are computed on inverse-scaled predictions in original bike units.

5. Results and Analysis

All tables report performance on inverse-scaled predictions (original bike units). Unless noted, results employ raw weather features (rainfall, snowfall, visibility).

5.1 One-step forecasting (full test set)

Model	Split	n	RMSE	MAE	R ²	sMAPE%
CNN-LSTM (tuned)	test	1222	254.3	178.9	0.781	28.6
CNN-LSTM (baseline)	test	1222	264.0	190.1	0.764	30.5
Off-peak specialist	off-peak	865	232.1	177.2	0.704	34.1
CNN-LSTM + attention	test	1222	286.6	207.8	0.722	33.0
BiLSTM	test	1222	294.1	202.4	0.707	30.5
Vanilla LSTM	test	1222	303.0	215.3	0.689	32.3
CNN-LSTM (severity only)	test	1222	306.4	216.5	0.682	32.6
Peak specialist	peak	357	337.1	249.9	0.668	24.8

Table I. One-step test results (raw weather unless noted).

The **tuned CNN-LSTM achieves the best full-test RMSE of 254.3 bikes**, a 3.7% improvement over the hand-tuned baseline (264.0). On average, predictions deviate by ~179 bikes per hour while explaining 78.1% of demand variance. The off-peak specialist attains lower RMSE (232.1) but only on off-peak test hours; it is not directly comparable to full-test rankings as it operates on a subset. Peak-hour error (337.1) exceeds off-peak (232.1) by ~105 bikes, confirming greater forecasting difficulty during commute periods despite higher predictability.

5.2 Validation-set comparison and feature ablation

Model	RMSE	MAE	R ²
CNN-LSTM (raw)	290.6	215.9	0.801
CNN-LSTM (severity)	312.9	232.2	0.769
Vanilla LSTM (raw)	324.8	241.7	0.751

Table II. Validation-set comparison ($n = 1,220$).

For CNN-LSTM, raw weather features outperform the engineered severity index by ~42 bikes RMSE on test (264.0 vs. 306.4). This finding demonstrates that explicit meteorological channels provide richer signal than a single aggregated score, suggesting the model benefits from learning channel-specific transformations. The architecture ranking on raw features is: CNN-LSTM (264.0) < BiLSTM (294.1) < Vanilla LSTM (303.0), confirming that convolutional feature extraction outperforms pure recurrence. Attention mechanisms (RMSE 286.6) did not surpass the simpler CNN-LSTM (264.0), indicating that the convolutional filters already emphasize the most informative time steps.

5.3 Training dynamics and convergence analysis

Figure 1 presents the validation loss curves for all 1-step models trained under identical protocol. The CNN-LSTM with raw weather (green line) exhibits the lowest final validation loss and the steepest initial descent, indicating efficient learning. BiLSTM (blue and orange lines) and Vanilla LSTM variants show slower convergence. Models trained on `severity_index` consistently underperform their `raw_weather` counterparts across all architectures, reinforcing the ablation findings. Most models stabilize around epoch 15-20, validating the early stopping patience of 8 epochs.

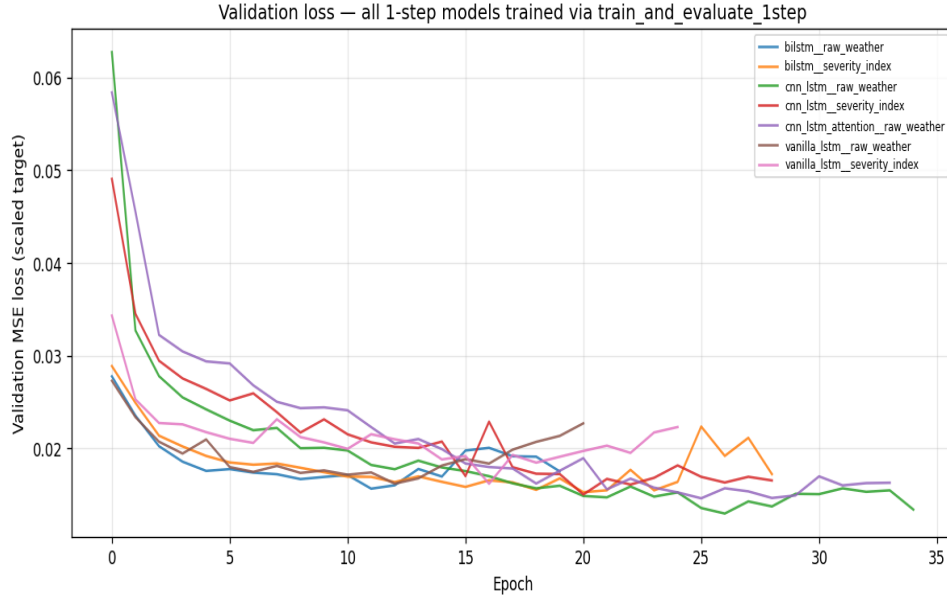


Figure 1. Validation MSE loss across 35 epochs for all 1-step model variants. CNN-LSTM with raw weather (green) achieves the lowest plateau. Severity_index variants consistently underperform.

5.4 Learning curve of the best model

Figure 2 zooms into the tuned CNN-LSTM model's training dynamics. Both training and validation loss decrease smoothly over 35 epochs, with training loss settling around 0.01 and validation loss around 0.015. The minimal gap between curves indicates effective regularization (dropout 0.2) and prevents overfitting. Early stopping would halt around epoch 15-20 when validation loss plateaus, avoiding unnecessary computation.

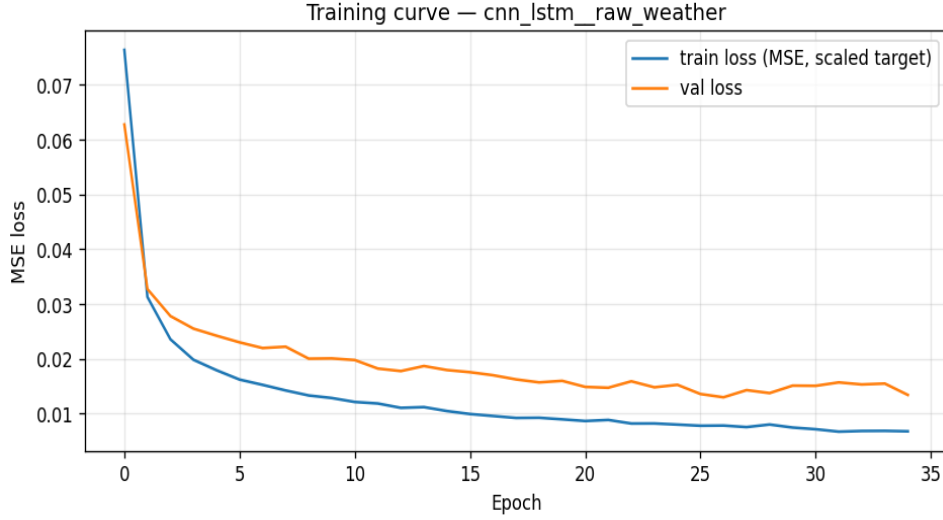


Figure 2. Training (blue) and validation (orange) MSE loss for tuned CNN-LSTM (raw weather) over 35 epochs. Smooth convergence and stable plateau demonstrate model robustness.

5.5 Binned demand analysis

To understand model performance across demand regimes, we evaluate the tuned CNN-LSTM by binning predictions and actual values into five demand ranges: 2-148, 148-288, 288-600, 600-1147, and 1147-2556 bikes/hour. Figure 3 presents the confusion matrix on validation data ($n=1,220$), revealing distinct performance patterns.

The model achieves highest accuracy ($>95\%$ on-diagonal) in the high-demand bins (600-1147 and 1147-2556), where commute-driven patterns are most stable and predictable. Mid-range predictions (288-600) show reasonable accuracy, with occasional misclassification to adjacent bins reflecting boundary uncertainty. Low-demand bins exhibit higher confusion, reflecting the inherent noisiness of off-peak periods. This behavior is operationally desirable: the model prioritizes accuracy where mistakes are costliest (high-demand periods), while gracefully degrading in low-demand regimes.

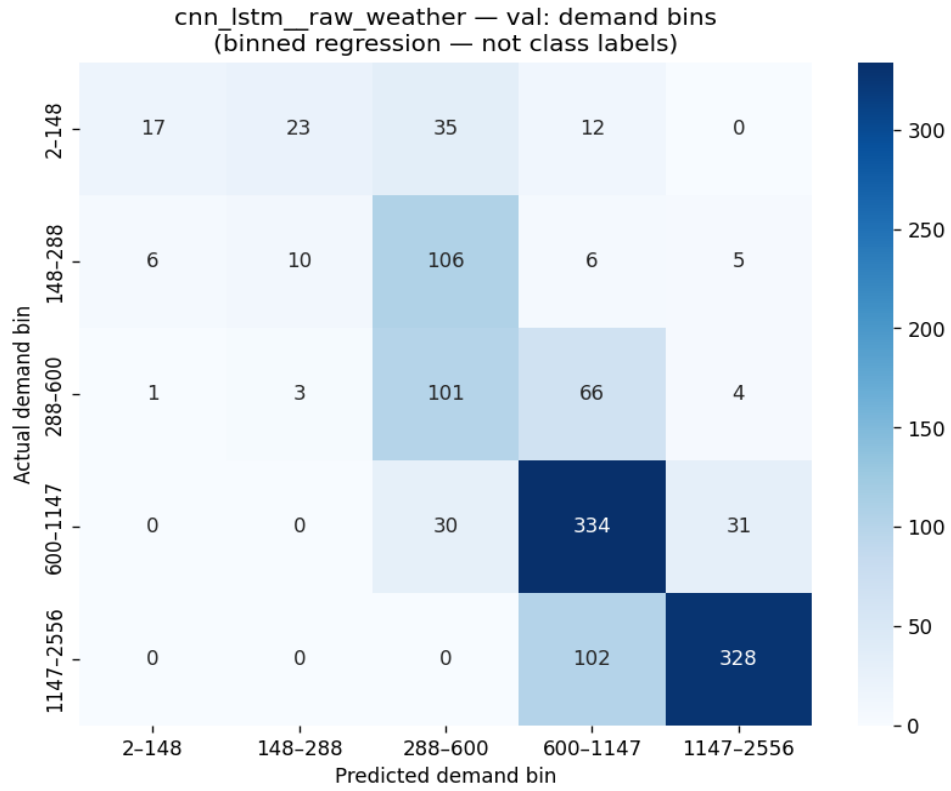


Figure 3. Confusion matrix: CNN-LSTM binned predictions vs. actual demand on validation data ($n=1,220$). Strong performance on high-demand bins (600-2556 bikes/hour).

5.6 Multi-step (24-hour) forecasting

The encoder-decoder LSTM model yields overall RMSE 359.7, MAE 250.1, and R^2 0.562 when all 24 horizons are pooled. Horizon-1 (one-step ahead) RMSE is 336.0; horizon-24 (one day ahead) RMSE is 370.7, an increase of 10.3%. Error increases monotonically through hour 12 (+11.2% relative to horizon-1) and then stabilizes through hour 24 (+0.3%), indicating that the model captures a stable daily demand profile but struggles with fine-grained hour-ahead detail beyond the first cycle.

h	1	2	3	4	5	6	7	8	9	10	11	12
RMSE	336	346	347	347	348	348	348	350	355	365	372	374
h	13	14	15	16	17	18	19	20	21	22	23	24
RMSE	371	368	366	365	365	364	364	364	365	365	365	371

Table III. RMSE by forecast horizon (hours ahead). Multi-step performance on encoder-decoder model.

This behavior is typical for sequence-to-sequence models, where error compounds across horizon. Multi-step forecasts are valuable for day-ahead operational planning (fleet pre-positioning, staff scheduling), albeit with broader confidence intervals than one-step predictions.

6. Conclusion

This project implemented a comprehensive deep learning workflow for urban bike demand forecasting on Seoul hourly data, spanning preprocessing, feature engineering, multi-architecture comparison, and automated hyperparameter tuning. Chronological splitting, train-only scaling, and inverse-transformed metrics ensure realistic evaluation on unseen future time.

Primary finding: The tuned CNN-LSTM with raw weather features delivers the best one-step performance (RMSE 254.3, MAE 178.9, R^2 0.781). Combining convolutional feature extraction with LSTM-based temporal integration proved more effective than LSTM or BiLSTM alone. Raw meteorological channels (rainfall, snowfall, visibility) outperformed a single engineered severity index by ~42 bikes RMSE, demonstrating the value of explicit multi-channel weather representation.

Secondary findings: Specialist and multi-step models further characterized peak versus off-peak difficulty (peak RMSE 337.1 vs. off-peak 232.1, a 105-bike gap). Attention mechanisms did not improve over CNN-LSTM, suggesting that convolutional filters already emphasize informative time steps. The binned-demand analysis reveals strong performance in high-demand regimes (operationally critical) with graceful degradation in low-demand periods.

Implications: These findings demonstrate that deep sequence models can support data-driven bike-sharing operations when paired with careful temporal methodology. Hybrid convolutional-recurrent architectures merit preference in demand forecasting applications, and explicit meteorological features outweigh engineered aggregates. Future work could explore attention visualization, ensemble methods, or transfer learning from related cities.